

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE COMPUTAÇÃO

MARCEL FERNANDO LOBO DE FEO
MATHEUS MATOS ANDRADE
PAULO DE MELO BORGES
SERGIO HENRIQUE RIBEIRO MATOS

RELATÓRIO DE INTELIGÊNCIA COMPUTACIONAL
Algoritmo Genético para problemas de Criptoaritmética

UBERLÂNDIA - MG
2026

INTRODUÇÃO

Este relatório documenta a modelagem, a otimização e a análise de generalização de um Algoritmo Genético (AG) aplicado à resolução de equações criptoaritméticas. O estudo foi conduzido em três etapas iterativas: a primeira estabeleceu um *baseline* comparando operadores básicos, identificando a superioridade do Crossover PMX e da Seleção por Roleta (51,2% de convergência). A segunda etapa realizou o refinamento de hiperparâmetros, atingindo o pico ótimo de 99,2% de convergência ao aplicar uma taxa de mutação agressiva (50%). Por fim, a terceira etapa aplicou essa configuração a cinco problemas distintos e investigou uma nova função de avaliação baseada no erro posicional.

Para garantir a reprodutibilidade das métricas de desempenho (especialmente o Tempo Médio de Execução), os experimentos foram processados em single-thread no ambiente Java sobre uma máquina equipada com processador AMD Ryzen 7 5700X (com overclock ativo para 4.85 GHz) e 32GB de memória RAM operando a 3200 MHz. Essa capacidade computacional justifica os baixos tempos absolutos de convergência registrados nos ensaios.

Os resultados mostraram que a configuração ótima para um problema específico não generalizou para os demais, sobretudo os de maior porte, e que a escolha da função de avaliação envolve um compromisso entre o desempenho médio (favorável à diferença global) e a uniformidade entre problemas (favorável ao erro posicional). Conclui-se que o desempenho do AG é limitado principalmente pelo tamanho do espaço de busca e pelo grau de isolamento da solução.

PRIMEIRA ETAPA

1. METODOLOGIA

A Etapa 1 avalia uma especificação base do AG sobre o problema **SEND + MORE = MONEY**, comparando variações de quatro operadores, sendo eles mutação, seleção, crossover e reinserção. Cada uma das 16 combinações possíveis foi executada 1000 vezes, totalizando 16.000 execuções. O ambiente experimental foi implementado em Java, com estrutura modular e foco em desempenho.

O **indivíduo** é representado por um vetor de inteiros de tamanho fixo 10, correspondente a uma permutação dos dígitos de 0 a 9. Cada letra do problema é mapeada para uma posição fixa do vetor, e seu dígito é lido diretamente dessa posição. A **função de avaliação (*fitness*)** adotada é a diferença global em valor absoluto entre os dois lados da operação: $|(SEND+MORE) - MONEY|$. Um *fitness* igual a zero indica solução válida.

1.1. Inversão do *fitness* para problema de otimização e Soluções Inválidas

Como é um problema de minimização, quanto menor o *fitness*, melhor o indivíduo. Esse aspecto exige atenção especial no operador de seleção, pois a roleta proporcional clássica assume o oposto. Foi, portanto, necessário inverter o *fitness* em uma aptidão no qual o menor erro corresponde ao maior peso na roleta. Adotou-se a inversão linear baseada no pior indivíduo da geração:

$$peso = (fitness_{max} + 1) - fitness,$$

em que o melhor indivíduo recebe o maior peso e o pior recebe peso unitário (nunca nulo, preservando diversidade). O operador de seleção por torneio não exige essa conversão, pois compara o *fitness* diretamente, vencendo o de menor valor.

Adicionalmente, atribuições com dígito zero na primeira letra de qualquer palavra violam a criptoaritmética. Dessa forma, aplicou-se uma penalização: indivíduos com zero à esquerda recebem um acréscimo elevado de *fitness*, garantindo que nunca sejam contabilizados como

solução válida, mas permanecendo na população para preservar material genético potencialmente útil.

A necessidade dessa penalização foi confirmada experimentalmente. Ao executar o AG sem a penalização, a taxa de convergência da melhor configuração saltou de cerca de 51% para mais de 80%. Esse aumento, porém, não era verdadeiro: ao decodificar as soluções encontradas, verificou-se que aproximadamente 93% delas eram inválidas, apresentando zero à esquerda. Tais soluções fecham a operação aritmeticamente, mas não constituem soluções legítimas do problema. No problema $SEND + MORE = MONEY$, a letra M tem o maior peso (a letra inicial da parcela e do resultado), de modo que atribuir $M = 0$ elimina de uma só vez a maior fonte de erro. Sem a penalização, o algoritmo explora esse atalho e converge para várias soluções com $M = 0$, como $6524 + 0735 = 07259$, que é aritmeticamente correta, mas em que as palavras MORE e MONEY perdem seu dígito inicial. A penalização acaba com esse atalho, assegurando que apenas soluções genuínas sejam contabilizadas como convergência.

1.2. Parâmetros e variações avaliadas

Os parâmetros fixos seguem a especificação do enunciado: **população de 100 indivíduos, 50 gerações, geração inicial aleatória sem repetições, mutação por troca de duas posições e taxa de *crossover* de 60%** (elevada para 80% no cenário de reinserção com elitismo). Sobre essa base, foram cruzadas as seguintes variações:

- **Taxa de mutação:** 10% (TM1) e 20% (TM2);
- **Seleção:** torneio com tour de 3 (S1) e roleta (S2);
- **Crossover:** cíclico (C1) e PMX (C2);
- **Reinserção:** ordenada - melhores entre pais e filhos - (R1) e pura com elitismo de 20% (R2)

As métricas coletadas foram o percentual de convergência e o tempo médio de execução. Para garantir reprodutibilidade e comparação justa, cada execução utiliza uma *seed fixa*, de modo que todas as configurações partem das mesmas populações iniciais.

2. RESULTADOS

A Tabela 1 apresenta o desempenho das 16 configurações sobre o problema **SEND + MORE = MONEY**. A linha destacada corresponde à melhor configuração.

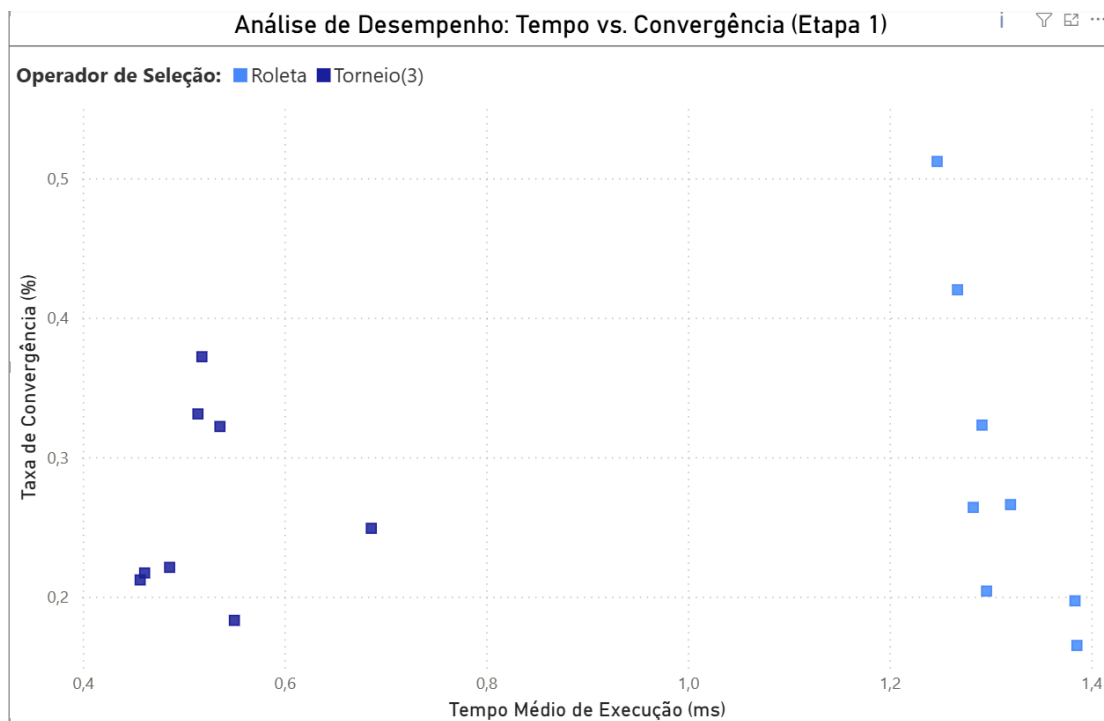
Tabela 1 - Desempenho das 16 configurações da Etapa 1.

Config.	Taxa Mut.	Seleção	Crossover	Reinserção	Conv. (%)	Tempo (ms)
1	10%	Torneio(3)	Cíclico	Ordenada	18,3	0,55
2	10%	Torneio(3)	Cíclico	Elitismo(20%)	21,7	0,46
3	10%	Torneio(3)	PMX	Ordenada	24,9	0,69
4	10%	Torneio(3)	PMX	Elitismo(20%)	33,1	0,51
5	10%	Roleta	Cíclico	Ordenada	16,5	1,39
6	10%	Roleta	Cíclico	Elitismo(20%)	19,7	1,38
7	10%	Roleta	PMX	Ordenada	26,6	1,32
8	10%	Roleta	PMX	Elitismo(20%)	42,0	1,27
9	20%	Torneio(3)	Cíclico	Ordenada	22,1	0,49
10	20%	Torneio(3)	Cíclico	Elitismo(20%)	21,2	0,46
11	20%	Torneio(3)	PMX	Ordenada	32,2	0,54
12	20%	Torneio(3)	PMX	Elitismo(20%)	37,2	0,52
13	20%	Roleta	Cíclico	Ordenada	20,4	1,30
14	20%	Roleta	Cíclico	Elitismo(20%)	26,4	1,28
15	20%	Roleta	PMX	Ordenada	32,3	1,29
16	20%	Roleta	PMX	Elitismo(20%)	51,2	1,25

3. ANÁLISE E VISUALIZAÇÃO

A análise dos resultados mostra alguns padrões. Primeiramente, todas as configurações com *PMX* superam suas equivalentes com *Crossover Cíclico*. Em segundo lugar, a reinserção com Elitismo supera consistentemente a reinserção ordenada quando combinada com *PMX*. Além disso, a taxa de mutação de 20% favorece as melhores configurações, ao introduzir maior diversidade e ajudar o algoritmo a escapar de mínimos locais. A combinação desses fatores (roleta, *PMX* e elitismo, com mutação de 20%) produz a configuração de melhor desempenho.

Além disso, embora a seleção por Roleta tenha apresentado maior taxa de convergência no geral, o Torneio demonstrou ser mais eficiente em termos de tempo médio.



Heatmap de Convergência -

Crossover	Elitismo(20%)	Ordenada	Total
Cíclico	0,89	0,77	1,66
PMX	1,64	1,16	2,80
Total	2,53	1,93	4,46

4. CONCLUSÃO E PRÓXIMA ETAPA

A melhor configuração da Etapa 1 é a **Configuração 16 - mutação 20%, seleção por roleta, *crossover* PMX e reinserção com elitismo de 20%**, com 51,2% de convergência e tempo médio de 1,25 ms.

Apesar de ser a melhor, essa configuração converge em pouco mais da metade das execuções. Na Etapa 2 a Configuração 16 será refinada por meio de variações de parâmetros, como tamanho de população, número de gerações, taxas de mutação e elitismo, com objetivo de elevar a convergência mas com um limite de acréscimo de 50% no tempo de execução.

SEGUNDA ETAPA

1. VARIAÇÕES AVALIADAS

Esta etapa parte da melhor configuração obtida na primeira etapa, a Configuração 16 (mutação de 20%, seleção por roleta, *crossover* PMX e reinserção com elitismo de 20%) e busca refiná-la por meio de variações de parâmetros.

Foram avaliadas 16 variações (V0 a V15), todas partindo da Configuração 16 e alterando um eixo por vez, para isolar o efeito de cada parâmetro; as demais (V9 a V15) combinam eixos que se mostraram benéficos isoladamente. A variação V0 reproduz a Configuração 16 e serve como *baseline* para o cálculo do acréscimo de tempo.

- **Número de gerações (V1, V2):** aumento de 50 para 75 e 100, concedendo mais tempo de busca
- **Tamanho da população (V3, V4):** aumento de 100 para 150 e 200, ampliando a diversidade
- **Taxa de mutação (V5, V6):** aumento de 20% para 30% e 40%, intensificando a exploração
- **Taxa de elitismo (V7, V8):** variação para 10% e 30%, testando o equilíbrio diversidade/intensificação

- **Combinações e Refinamento (V9 a V15):** junção dos eixos benéficos (população, gerações e mutação) e exploração de taxas de mutação maiores combinadas a mais gerações.

2. RESULTADOS

A Tabela 2 apresenta o desempenho das 16 variações sobre o problema SEND + MORE = MONEY (1000 execuções cada). A linha destacada corresponde à melhor variação. As linhas em vermelho estouraram o limite de 50% de acréscimo de tempo.

Tabela 2 - Desempenho das 12 variações da Etapa 2.

Var.	Descrição	Pop	Ger.	Mut.	Elit.	Conv. (%)	Tempo (ms)	Acrésc. (%)
V0	Baseline (Config 16)	100	50	20%	20%	48,5	1,34	+0,0
V1	Gerações 75	100	75	20%	20%	54,5	1,59	+18,9
V2	Gerações 100	100	100	20%	20%	50,8	2,00	+49,0
V3	População 150	150	50	20%	20%	64,9	2,02	+50,7
V4	População 200	200	50	20%	20%	76,6	3,01	+124,8
V5	Mutação 30%	100	50	30%	20%	62,2	1,24	-7,7
V6	Mutação 40%	100	50	40%	20%	86,8	1,14	-14,9
V7	Elitismo 10%	100	50	20%	10%	67,3	1,32	-1,6
V8	Elitismo 30%	100	50	20%	30%	47,2	1,19	-11,3
V9	Pop 150 + Gen 75	150	75	20%	20%	64,9	2,56	+91,3
V10	Pop 150 + Mut 30%	150	50	30%	20%	74,9	2,00	+49,6
V11	Gen 75 + Mut 30%	100	75	30%	20%	66,9	1,55	+16,0
V12	Mutação 50%	100	50	50%	20%	92,8	1,15	-13,9
V13	Mut 40% + Elit 10%	100	50	40%	10%	89,1	1,33	-0,9
V14	Mut 40% + Gen 75	100	75	40%	20%	95,6	1,22	-9,2
V15	Mut 50% + Gen 75	100	75	50%	20%	99,2	1,15	-13,9%

É importante observar que a taxa de convergência da V0 (48,5%) difere da obtida pela Configuração 16 (51,2%) na etapa anterior, embora ambas utilizem exatamente os mesmos parâmetros. Essa diferença é esperada do ponto de vista estatístico, pois os experimentos da segunda etapa foram executados com uma nova *seed*, gerando uma sequência diferente de números aleatórios e, conseqüentemente, populações iniciais distintas.

Como algoritmos genéticos são métodos estocásticos, pequenas variações entre conjuntos independentes de execuções são naturais, mesmo mantendo a mesma configuração. O fato de os resultados permanecerem próximos demonstra que o comportamento do algoritmo é consistente e que o código não apresenta qualquer viés em favor de uma configuração específica, refletindo apenas a variabilidade inerente ao processo evolutivo.

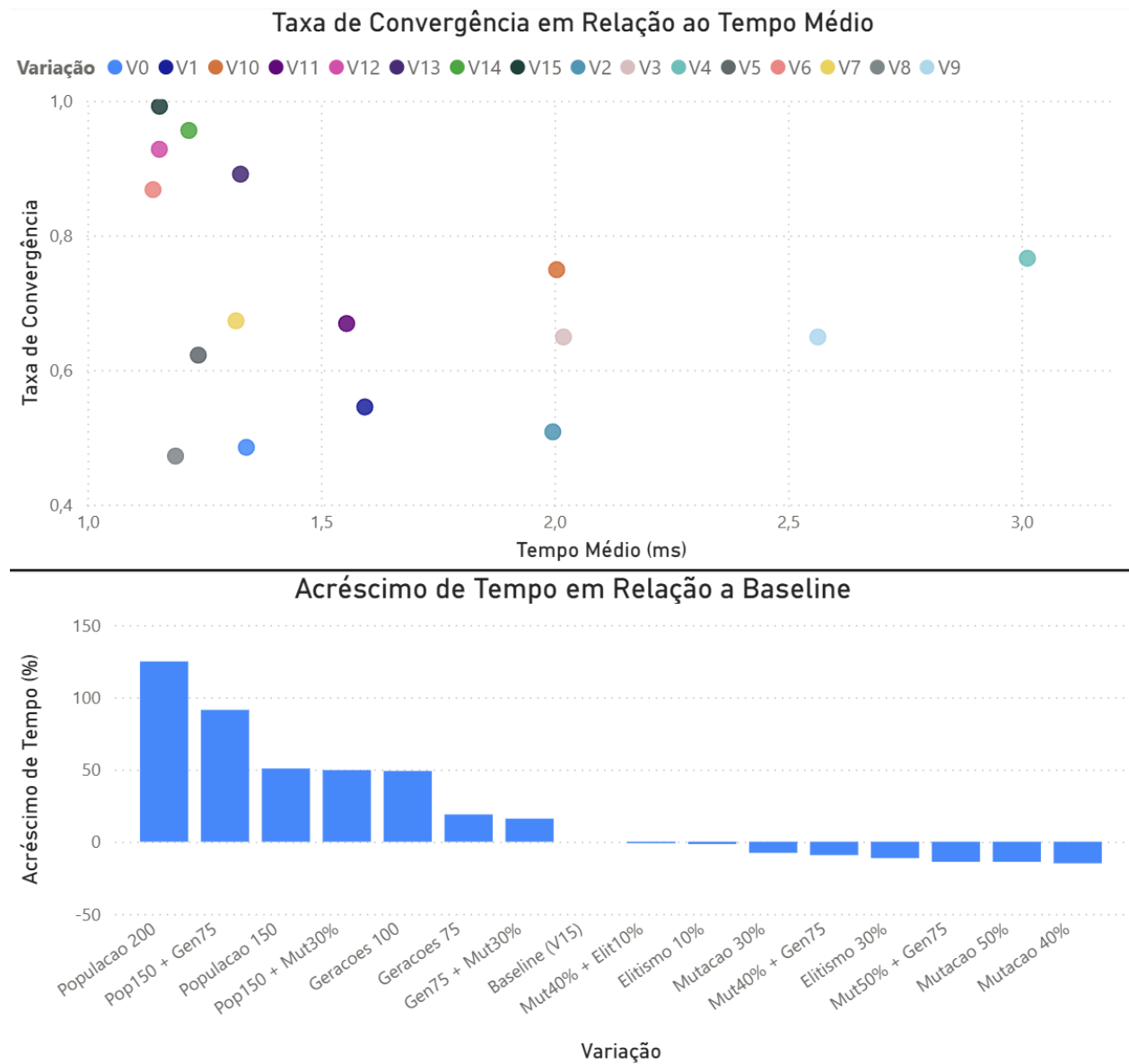
3. ANÁLISE E VISUALIZAÇÃO

O resultado mais expressivo foi o da taxa de mutação. O aumento progressivo de 20% para 30% (V5) e 40% (V6) elevou a convergência de forma consistente, atingindo 86,8% de convergência na V6 - um salto substancial em comparação com os 48,5% do *baseline*. Mais relevante ainda, a V6 não aumentou o tempo de execução: ao contrário, reduziu-o em 14,9% (de 1,34 ms para 1,14 ms). A explicação é que, com mais exploração, o algoritmo encontra a solução em menos gerações e encerra antes pelo critério de parada, diminuindo o tempo total apesar de cada geração ser igualmente custosa.

O aumento da população (V3, V4) também elevou a convergência, mas a um custo de tempo crescente. A V4 (população 200) atingiu 76,6%, porém com acréscimo de 124,8% no tempo, muito acima do limite de 50%, tornando-a inelegível. A V3 (população 150) ficou no limiar (+50,7%), mas com convergência bem inferior à da V6, a um custo de tempo muito maior. Já a variação do elitismo mostrou que o elitismo excessivo diminui a diversidade e favorece ficar preso em mínimos locais: reduzi-lo para 10% (V7) melhorou o desempenho para 67,3%, enquanto aumentá-lo para 30% (V8) o reduziu a 47,2%.

As combinações iniciais (V9 a 11) não superaram a melhor variação individual: as que ampliaram a população (V9 e V10) ultrapassaram (ou quase ultrapassaram) o limite de tempo

(+91,3% e 49,6%), enquanto a V11 permaneceu elegível (+16,0%) mas com convergência inferior. Esses resultados motivaram uma segunda rodada de refinamento, concentrada no eixo mais promissor, a mutação. Elevar a taxa para 50% (V12) aumentou a convergência para 92,8%, indicando que o ganho com mutação não havia se esgotado em 40%. Combinar mutação alta com mais gerações foi ainda mais eficaz: a V14 (mutação 40% + 75 gerações) atingiu 95,6% e a V15 (mutação 50% + 75 gerações) alcançou 99,2% de convergência. Todas essas variações mantiveram tempo inferior ao *baseline*, pois convergem em poucas gerações e encerram cedo. Isso confirma que, nessa etapa e considerando apenas o “SEND + MORE = MONEY”, intensificar a exploração foi a direção mais eficiente.



4. CONCLUSÃO E COMPARAÇÃO COM PRIMEIRA ETAPA

A melhor variação da segunda etapa é a **V15 (taxa de mutação de 50%, com 75 gerações)**, com 99,2% de convergência. Ela tem a maior convergência entre todas as variações avaliadas e, ainda assim, executa em tempo inferior ao do *baseline*.

Em comparação com a primeira etapa, a convergência saltou de 51,2% (Configuração 16) para 99,2% (V15), um ganho de 48 pontos percentuais sem custo de tempo adicional. O AG, que

antes resolvia o problema em pouco mais da metade das execuções, passou a resolvê-lo em praticamente todas.

A próxima etapa avalia a capacidade de generalização do algoritmo, aplicando a melhor configuração obtida a um conjunto de cinco problemas distintos de criptoaritmética e investigando uma nova função de avaliação baseada no erro posicional, entre outras novas variações.

TERCEIRA ETAPA

1. GENERALIZAÇÃO DA MELHOR CONFIGURAÇÃO DA ETAPA 2

Essa etapa avalia a capacidade de generalização do Algoritmo Genético: seu desempenho não apenas no problema usado durante o ajuste (**SEND + MORE = MONEY**), mas em um conjunto de cinco problemas distintos de criptoaritmética, de complexidades variadas. Os problemas avaliados foram: **SEND + MORE = MONEY**, **EAT + THAT = APPLE**, **CROSS + ROADS = DANGER**, **COCA + COLA = OASIS** e **DONALD + GERALD = ROBERT**, com 8 a 10 letras distintas e resultados de 5 a 6 dígitos.

O primeiro experimento aplica a melhor configuração obtida na segunda etapa (a variação V15) a cada um dos cinco problemas, com 1000 execuções por problema. Vale destacar que para esse experimento foi utilizada outra *seed*.

Tabela 3 - Desempenho da configuração V15 em cada problema

Problema	Conv. (%)	Tempo (ms)
SEND + MORE = MONEY	98,3	1,33
EAT + THAT = APPLE	99,3	0,91
CROSS + ROADS = DANGER	2,6	3,09
COCA + COLA = OASIS	29,0	2,44
DONALD + GERALD = ROBERT	8,4	3,04
Média	47,5	2,16

Os resultados revelam que a configuração V15, embora convergisse em 99,2% no problema usado para o ajuste, não generaliza de maneira uniforme. Ela mantém o desempenho elevado em **SEND + MORE = MONEY** (98,3%) e em **EAT + THAT = APPLE** (99,3%), mas cai drasticamente nos problemas maiores: 29,0% em **COCA + COLA = OASIS**, 8,4% em **DONALD + GERALD = ROBERT** e apenas 2,6% em **CROSS + ROADS = DANGER**. A média geral é de

47,5%.

Observa-se ainda que os problemas de menor convergência são também os de maior tempo de execução. Isso ocorre porque, quando o AG não encontra a solução, ele percorre todas as gerações até o limite (gastando o tempo máximo). Esses resultados indicam que a configuração estava **fortemente ajustada** ao problema da segunda etapa e motivam a busca, a seguir, por uma versão mais genérica do algoritmo, incluindo a avaliação de uma nova função de aptidão.

2. FUNÇÃO DE ERRO POSICIONAL

A função de diferença global, usada nas etapas anteriores, mede o erro como a distância numérica entre dois lados da operação. Nessa abordagem, um erro em um dígito de ordem alta (por exemplo, o dígito de milhar) pesa mais do que um erro em um dígito de ordem baixa, pois é multiplicado pela casa decimal correspondente.

Já a função de erro posicional avalia a operação coluna a coluna, da direita para esquerda. Para cada coluna, soma os dígitos das parcelas (mais o transporte da coluna anterior) e compara o dígito das unidades resultantes com o dígito esperado no resultado, acumulando a diferença absoluta. O transporte (carry) é propagado entre colunas, de modo que a função permanece fiel à aritmética da soma. A diferença essencial em relação à função global é que, no erro posicional, **cada coluna tem o mesmo peso**, um erro nas unidades conta tanto quanto um erro nos milhares. A restrição de zero à esquerda é tratada por uma penalização calibrada a um valor menor, compatível com a escala reduzida do erro posicional.

3. VARIAÇÕES AVALIADAS E RESULTADOS

Para buscar uma versão mais genérica do algoritmo, foram avaliadas variações que substituem a função de avaliação pela de erro posicional, além de testar diferentes parâmetros e operadores. A Tabela 4 apresenta a taxa de convergência de cada variação em cada um dos cinco problemas, bem como a média.

Tabela 4 - Taxa de convergência por problema e média dos cinco problemas

Var.	Descrição	P1 Conv. (%)	P2 Conv. (%)	P3 Conv. (%)	P4 Conv. (%)	P5 Conv. (%)	Média (%)	Tempo (ms)
V0	Baseline V15	98,3	99,3	2,6	29,0	8,4	47,5	2,16
V1	Erro posicional	17,2	35,6	6,1	34,8	11,8	21,1	2,88
V2	Posicional + Mut. 60%	20,1	35,1	7,8	43,2	14,4	24,1	2,86
V3	Posicional + Torneio	11,4	27,9	4,7	16,1	8,6	13,7	1,53
V4	Posicional + Cross. Cíclico	13,8	26,6	5,5	25,9	9,9	16,3	2,79
V5	Global + Ger. 110	99,8	99,8	2,5	31,0	14,2	49,5	2,94
V6	Global + Pop. 150	99,9	99,9	3,6	38,1	14,1	51,1	3,88
V7	Global + Pop. 200 + Ger. 100	100,0	100,0	3,7	48,7	22,0	54,9	7,65

P1: “SEND+MORE=MONEY”

P2: “EAT+THAT=APPLE”

P3: “CROSS+ROADS=DANGER”

P4: “COCA+COLA=OASIS”

P5: “DONALD+GERALD=ROBERT”

4. ANÁLISE E VISUALIZAÇÃO

Para buscar uma versão mais genérica do algoritmo, foram avaliadas variações que substituem a função de avaliação pela de erro posicional, além de testar diferentes parâmetros e operadores. A Tabela 4 apresenta a taxa de convergência de cada variação em cada um dos cinco problemas, bem como a média.

Na média dos cinco problemas, a função global (47,5%) supera a melhor variação posicional (24,1%, V2). A função global é bastante desigual (excelente em dois problemas, fraca em um e muito fraca em dois), ao passo que a função posicional apresenta desempenho mais uniforme entre os problemas. No problema COCA + COLA = OASIS, inclusive, a melhor variação posicional (43,2%) supera a função global (29,0%).

Por outro lado, a função posicional reduziu drasticamente a convergência nos problemas mais simples. Em SEND + MORE = MONEY, por exemplo, a taxa caiu de 98,3% para apenas 20,1% na melhor configuração (V2). Isso sugere que, para esse problema, a diferença global fornece um gradiente de busca mais eficiente, conduzindo o algoritmo até a solução com maior facilidade.

As demais modificações também produziram efeitos previsíveis. A substituição da seleção por roleta pelo torneio (V3) reduziu ainda mais a convergência média, indicando que, juntamente com a função posicional, a seleção proporcional favorece melhor a exploração do espaço de busca. Da mesma forma, a troca do crossover PMX pelo crossover cíclico (V4) apresentou pior desempenho médio, confirmando novamente os resultados observados nas etapas anteriores.

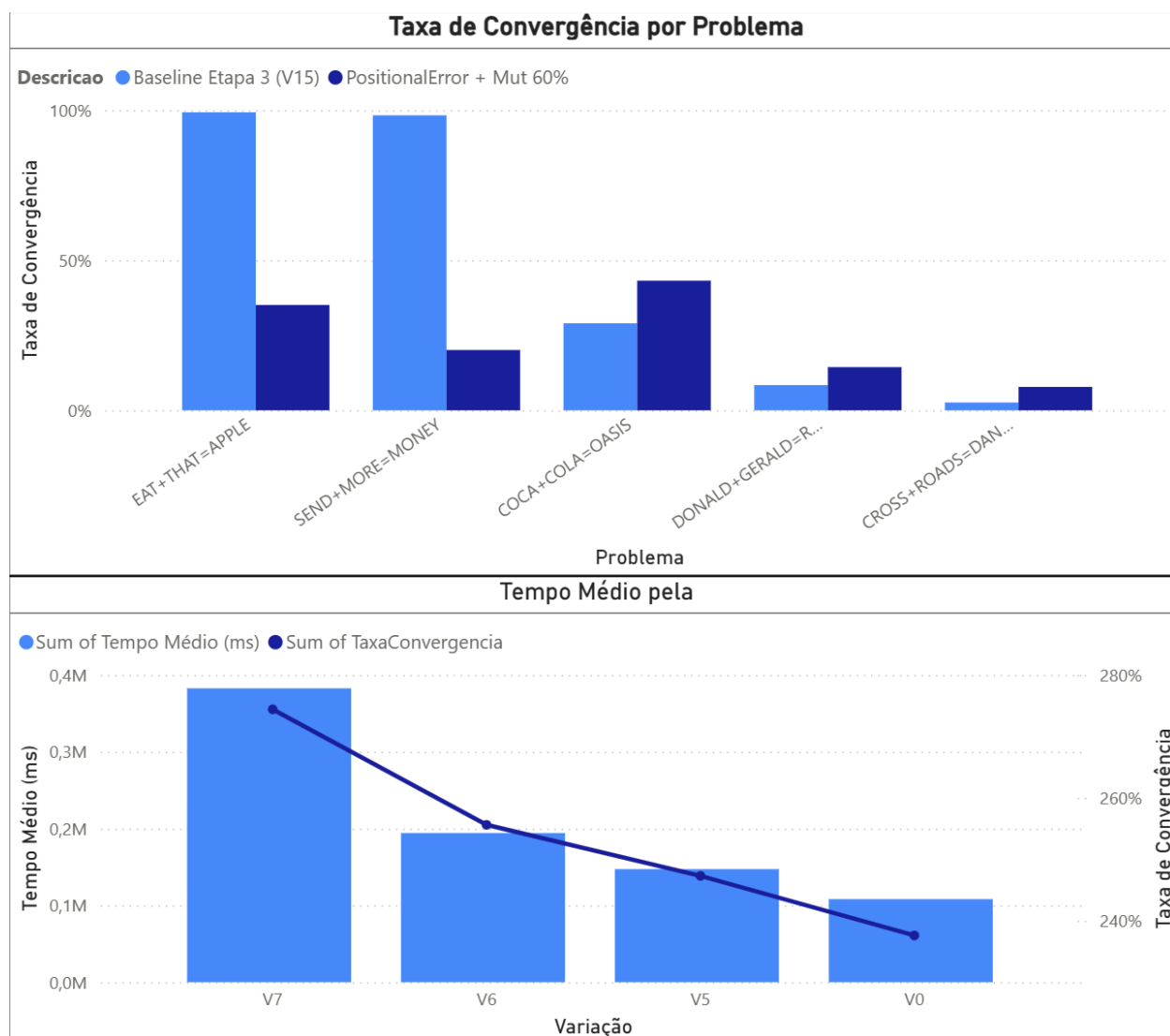
Entre as configurações que mantiveram a função de avaliação global, o aumento do número de gerações (V5) promoveu pequeno ganho na média geral (49,5%). O aumento da população para 150 indivíduos (V6) elevou essa média para 51,1%, enquanto a combinação de população de 200 indivíduos com 100 gerações (V7) atingiu a maior convergência média de toda a etapa (54,9%). Entretanto, essa melhoria veio acompanhada de um aumento expressivo no tempo médio de execução (7,65 ms), mais que o triplo do tempo da configuração *baseline*, evidenciando o custo computacional necessário para obter esse ganho adicional.

De maneira geral, os experimentos mostram que a principal limitação do algoritmo não está apenas na função de avaliação utilizada, mas também na elevada complexidade dos problemas maiores. Embora o erro posicional tenha apresentado desempenho mais equilibrado entre diferentes instâncias, ele não foi suficiente para superar a função global em termos de convergência média. Por outro lado, aumentar os recursos computacionais (população e gerações) permitiu melhorar a capacidade de generalização do algoritmo, ainda que com custo significativo de processamento.

Essa limitação frente aos problemas maiores pode ser explicada por dois fatores que se reforçam: o tamanho do espaço de busca e o isolamento da solução dentro desse espaço. O tamanho do espaço de busca depende do número de letras distintas na equação: o problema $EAT + THAT = APPLE$ possui apenas 6 letras distintas, resultando em $10 \times 9 \times 8 \times 7 \times 6 \times 5 = 151.200$ permutações possíveis. Nesse espaço relativamente pequeno, as avaliações do algoritmo cobrem uma fração significativa do domínio, e a convergência é alta (99,3%). Já os problemas $CROSS + ROADS = DANGER$ (9 letras) e $DONALD + GERALD = ROBERT$ (10 letras) possuem maior espaço ($10! = 3.628.800$ combinações), o que reduz a chance do algoritmo encontrar a solução no número de gerações disponível.

O tamanho do espaço, porém, não explica sozinho a dificuldade. Analisando a distribuição de *fitness* de cada problema observa-se que a dificuldade depende também de quão “isolada” está a solução (quantas soluções parciais existem ao seu redor para guiar a busca). Os problemas $CROSS + ROADS = DANGER$ e $DONALD + GERALD = ROBERT$ são os mais isolados e, também, de pior convergência.

Além disso, o isolamento é agravado pela representação por permutação. Como cada dígito só pode ser usado uma vez, corrigir uma letra de casa decimal alta para reduzir o erro frequentemente obriga a trocar um dígito que já estava correto em outra posição, desfazendo parte do progresso anterior. Esse efeito dificulta a melhoria gradual e “aprisiona” o algoritmo em mínimos locais, fazendo a taxa de convergência cair.



5. CONCLUSÃO

Os resultados da terceira etapa mostram que a configuração otimizada na etapa anterior não manteve o mesmo desempenho quando aplicada a diferentes problemas de criptoaritmética. Apesar de ter obtido excelentes resultados nos problemas mais simples, sua convergência caiu significativamente nos problemas mais complexos.

A função de erro posicional apresentou um comportamento mais equilibrado entre os diferentes problemas, no entanto, sua convergência média foi inferior à da função de erro global.

Já o aumento da população e do número de gerações melhorou a média de convergência, porém com aumento do tempo de execução.

Assim, conclui-se que a melhor configuração para um problema específico não é necessariamente a melhor para todos os casos. Os experimentos mostraram a importância de avaliar o algoritmo em diferentes instâncias para verificar sua capacidade de generalização e identificar o equilíbrio entre desempenho e custo computacional.

CONCLUSÃO

Este relatório documentou o desenvolvimento e a otimização de um Algoritmo Genético (AG) aplicado à resolução de equações criptoaritméticas. Ao longo de três etapas, foi possível evoluir de uma configuração inicial modesta para uma solução altamente eficiente (V15), atingindo 99,2% de convergência no problema base (*SEND+MORE=MONEY*).

A análise demonstrou que a otimização bem-sucedida para um problema específico não garante o mesmo sucesso frente a outros problemas. Embora o aumento de recursos computacionais (população e gerações) e o ajuste fino de operadores tenham superado limitações locais, problemas de maior complexidade revelaram que o desempenho do AG é severamente condicionado pelo tamanho do espaço de busca e pelo grau de isolamento das soluções.

Conclui-se, portanto, que a busca por uma configuração universal é um desafio complexo. O estudo evidenciou que a escolha da função de avaliação e dos parâmetros evolutivos deve equilibrar três pilares: a convergência específica, a capacidade de generalização para problemas distintos e o custo computacional. O sucesso na aplicação de algoritmos evolutivos, nesse contexto, reside menos na busca por uma "configuração perfeita" e mais na compreensão das propriedades do espaço de busca de cada problema, permitindo o ajuste consciente entre desempenho e viabilidade técnica.

REFERÊNCIAS

MARTINS, L. G. A. **Notas de aula da disciplina de Inteligência Computacional**. Faculdade de Computação (FACOM), Universidade Federal de Uberlândia (UFU), Uberlândia, 2026. Material não publicado.

GABRIEL, P. H. R.; DELBEM, A. C. B. **Fundamentos de algoritmos evolutivos**. São Carlos: ICMC-USP, 2008.

GOLDBERG, D. E. **Genetic Algorithms in Search, Optimization, and Machine Learning**. 1. ed. Reading, MA: Addison-Wesley, 1989.

HOLLAND, J. H. **Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence**. Ann Arbor, MI: University of Michigan Press, 1975.

LINDEN, R. **Algoritmos Genéticos: Um Importante Ramo da Inteligência Computacional**. 3. ed. Rio de Janeiro: Brasport, 2008.

MITCHELL, M. **An Introduction to Genetic Algorithms**. Cambridge, MA: MIT Press, 1998.